LCS, Greedy

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 29, 2024
Studiato	Done

Ricordiamo la definizione ricorsiva del problema:

$$c[i,j] = \begin{cases} 0 & i = 0, j = 0 \\ 1 + c[i-1, j-1] & i,j > 0 \wedge x_i = y_j \\ \max\{\, c[i, j-1], c[i-1, j] \,\} & i,j > 0 \wedge x_i \neq y_j \end{cases}$$

con $i \in [0, m]$ e $j \in [0, n]$.

Lo spazio dei sottoproblemi di LCS ha **dimensione polinomiale**. In particolare, lo spazio dei sottoproblemi ha dimensione $\Theta(m \cdot n)$, infatti è necessario calcolare $c[i,j]$ per $0 \leq j \leq n$ e per $0 \leq i \leq m$. Possiamo applicare la programmazione dinamica.

Scriviamo la procedura $\mathrm{LCS\text{-}Length}$, che prende in input due sequenze $X =< x_1, ..., x_m >$ e $Y =< y_1, ..., y_n >$ e le lunghezze m, n delle sequenze. Tale sequenza memorizza i valori $c[i,j]$ in una tabella. Tale tabella c avrà $m+1$ righe ed $n+1$ colonne, e calcola gli elementi in ordine di riga crescente da sinistra a destra. Mantiene pure la tabella b di dimensione $m \times n$ i cui elementi sono "**frecce**". Il tempo di questa procedura è $\Theta(n \cdot m)$, dato che ogni elemento è calcolato in $\Theta(1)$.

```
LCS-Length(X, Y, m, n):
    c = new-matrix((m+1) x (n+1)) [0:m, 0:n]
    b = new-matrix(m, n) [1:m, 1:n]
    for i = 0 to m do
        c[i,0] = 0
```

```
    for j = 0 to n do
        c[0,j] = 0
    for i = 1 to m do
        for j = 1 to n do
            if x[i] = y[j] then // consideriamo 1 + la soluzione al sottoproblema c[i-1, j-
                c[i,j] = c[i-1, j-1] + 1
                b[i,j] = freccia in alto a sinistra
            else if c[i-i, j] >= c[i, j-1] then
                c[i,j] = c[i-1, j]
                b[i,j] = freccia in alto
            else do
                c[i,j] = c[i, j-1]
                b[i,j] = freccia a sinistra
        // prendiamo il massimo tra c[i-1, j] e c[i, j-1] e lo poniamo in c[i,j]
    return c, b
```

Possiamo ottenere:

- la lunghezza della LCS prendendo l'elemento in basso a destra della matrice c;
- la LCS partendo dall'elemento in basso a destra della matrice c e seguendo le frecce della matrice b.

Da cui la funzione Print-LCS:

```
Print-LCS(b, X, i, j):
    if i = 0 or j = 0 then
        return
    if b[i,j] = freccia in alto a sinistra
        Print-LCS(b, X, i-1, j-1)
        print X[i]
    else if b[i,j] = freccia in alto
        Print-LCS(b, X, i-1, j)
    else if b[i,j] = freccia a sinistra
        Print-LCS(b, X, i, j-1)
```

che impiega $\Theta(m+n)$ perché il percorso più lungo è dato procedendo "a scala", quindi abbiamo $m+n$ passi. Possiamo migliorare il codice eliminando la tabella b, poiché $c[i,j]$ dipende solo da $c[i-1,j-1]$, $c[i-1,j]$ e da $c[i,j-1]$. Dato $c[i,j]$, si può determinare in tempo $\Theta(1)$ quale dei tre valori è stato usato per $c[i,j]$. Si può ricostruire una LCS in tempo $\Theta(n+m)$ usando una procedura simile a Print-LCS risparmiando uno spazio $\Theta(m \cdot n)$, ma lo spazio usato non diminuisce asintoticamente perché comunque la tabella c richiede $\Theta(m \cdot n)$ spazio.

Possiamo risparmiare asintoticamente lo spazio utilizzando esattamente due righe: la riga attuale e quella precedente. Lo spazio diventa $\Theta(2 \cdot n)$ a fronte di poter risolvere solamente il problema della lunghezza: **non è possibile, infatti, ricostruire la sequenza**.

Greedy

Un algoritmo greedy ottiene una soluzione ottima ad un problema di ottimizzazione attraverso una serie di scelte localmente ottime. Ad ogni "bivio" l'algoritmo fa la scelta che sembra migliore al momento. Si tratta di una strategia euristica che **non sempre** produce una soluzione ottima.

Affinché un problema sia risolvibile bisogna:

- formulare il problema di ottimizzazione come un problema nel quale **si fa una scelta e si rimane con un unico sottoproblema**, e di conseguenza **decidere l'euristica** (le scelte);
- dimostrare che **esiste sempre una soluzione ottima che contiene la scelta greedy** così che eseguire la scelta greedy è sicuro;
- verificare che tale problema **goda della proprietà di sottostruttura ottima**.

Proprietà di scelta greedy

La proprietà di scelta greedy vale se esiste una soluzione ottima S^* tale che S^* contiene la scelta greedy ad ogni iterazione. Tale proprietà ci assicura di poter assemblare una soluzione ottima globale facendo scelte greedy che sono localmente ottime.

> *Un problema di ottimizzazione soddisfa la proprietà di scelta greedy se esiste una soluzione S^* ottima globale ottenuta compiendo scelte localmente ottime ad ogni iterazione.*

Non dobbiamo mostrare che tutte le soluzioni ottime contengono le scelte greedy. Basta quindi provare che presa una soluzione ottima:

- essa contiene solo scelte greedy;
- non ne contiene una.

In quest'ultimo caso possiamo modificare tale scelta e sostituirla con quella greedy e dimostrare che la soluzione è ancora ottima. Generalmente, quindi, un problema ha la proprietà di scelta greedy se esaminando una soluzione globale e mostrando come modificarla sostituendo la scelta greedy a qualche altra scelta si ottiene una soluzione che è ancora ottima.

La scelta fatta da un algoritmo greedy può dipendere dalle scelte precedenti ma non da quelle successive.

Differenze con la programmazione dinamica

L'approccio greedy è un approccio **top-down**, mentre la programmazione dinamica è bottom-up.

Poiché la proprietà greedy è necessaria per i problemi risolubili con l'approccio greedy, possiamo dire che l'insieme di problemi che si possono risolvere con la tecnica greedy è incluso nell'insieme di problemi risolubili tramite programmazione dinamica, in quanto questa richiede solo che il problema goda della proprietà di sottostruttura ottima.

Con la programmazione dinamica risolviamo sottoproblemi prima di fare le scelte, e sfruttiamo la memorizzazione per evitare nuovamente il calcolo della ricorsione.

Semplificazione della proprietà di sottostruttura ottima

Una volta dimostrata la proprietà di scelta greedy, possiamo semplificare la proprietà di sottostruttura ottima. Infatti, basta soltanto mostrare **che combinando una soluzione ottima ad un sottoproblema a una scelta greedy si ottiene una soluzione globalmente ottima.**